III

UNIT III · CIE-1 PORTION

Supervised Learning: Classification & Regression

- **Ch 10 · Classification vs Regression** — CR's question 1
- **Ch 11 · Linear Regression (with worked examples)** — CR's question 3, plus polynomial regression
- **Ch 12 · Naive Bayes Classifier** — CR's question 2
- **Ch 13 · Safety net**: Logistic Regression, SVM, KNN — one page each, in case the paper strays from the 3 questions

For CIE-1 your CR shared these 3 topics from Unit III: (1) difference between classification and regression, (2) Naive Bayes classifier, (3) linear regression with example. Master Ch 10–12 completely; skim Ch 13.

10 Classification vs Regression ★ CIE-1 Q

Both are **supervised learning**: we learn from examples that already have the correct answer (label). The only difference is **what kind of answer** we predict.

CLASSIFICATION

Supervised learning that predicts a categorical class label from input features. Answers "**Which category?**". Example: classify an email as spam or not spam; pass / fail.

REGRESSION

Supervised learning that predicts a numerical (continuous) target from input features. Answers "**How much? How many?**". Example: predict house price from area and location; exam marks.

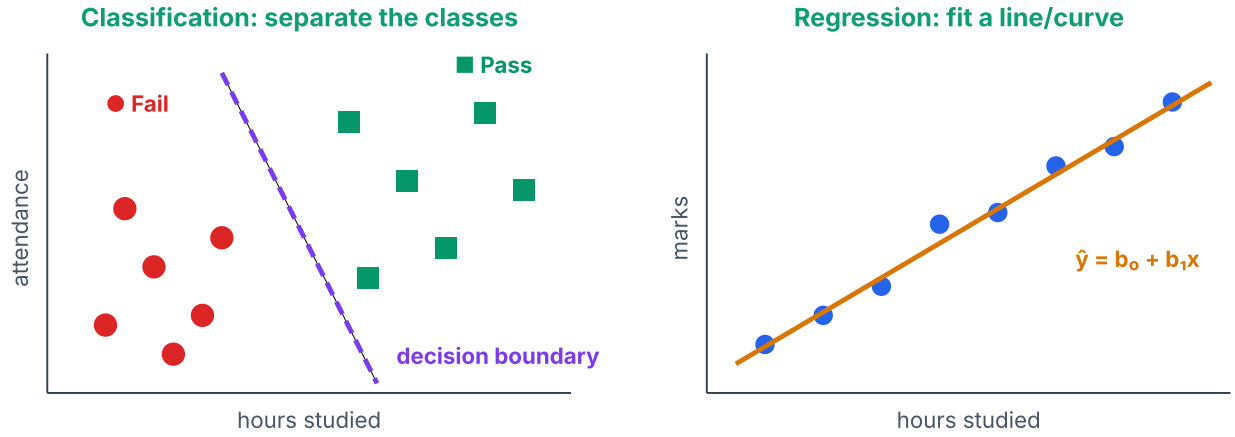


Fig 10.1 Classification outputs a class (which side of the boundary?). Regression outputs a number (where on the line?).

10.1 The complete comparison table ★ Very likely

Aspect	Classification	Regression
Objective	Predict a class or category	Predict a numerical magnitude
Target / output variable	Discrete / categorical (binary, multiclass, multilabel, ordinal)	Numeric, usually continuous
Output	A label, often with class probabilities	An estimated real-valued response
Question answered	"Which category?"	"How much / how many?"
Example: students	Predict pass / fail	Predict examination marks
Example: healthcare	Disease present / absent	Blood-glucose measurement
Example: real estate	Budget / mid-range / premium category	Selling price
Common algorithms	Logistic regression, Naive Bayes, SVM classifier, decision tree, random forest, k-NN	Linear, polynomial, ridge & lasso regression, SVR, regression tree, random-forest regressor
Learning loss	Cross-entropy (log loss), hinge loss	Squared error, absolute error
Evaluation metrics	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix	MAE, MSE, RMSE, R^2
Plots	Decision boundary, confusion matrix, ROC/PR curve	Fitted line/curve, residual plot, actual vs predicted
What an error means	Wrong class or inaccurate class probability	Difference between observed and predicted values
Decision threshold	Often converts a probability to a label (e.g. 0.5)	Usually not needed
Data concerns	Class imbalance, mislabelled samples	Outliers, unequal coverage of the numeric range
Interpretation	Class membership and probability	Magnitude, slope and change in the target
Applications	Spam, fraud, sentiment, image recognition, diagnosis	Price, sales, rainfall, demand forecasting, weather

10.2 The mathematical models side by side

Binary classification (logistic model):

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

output squashed into 0…1, then thresholded

Linear regression:

$\hat{y} = w^T x + b \;\; (= b_0 + b_1 x$ for one feature$)$

output is any real number

TRAP QUESTION

Logistic regression is a classification method despite its name — its output is a **class probability** which is then turned into a class by a threshold. (Why "regression"? It first models a continuous value — the probability $0 \le P(y=1) \le 1$ — and then a threshold converts it: P = 0.82 → class 1; P = 0.23 → class 0.)

10.3 Types of classification & quick ways to tell them apart

- **Binary**: 2 classes (Yes/No, Spam/Not spam, Fraud/Not fraud). **Multiclass**: > 2 classes, exactly one per example (Iris: Setosa / Versicolor / Virginica). **Multilabel**: an example can have several labels (a movie is both comedy and romance). **Ordinal**: ordered categories (low / medium / high).
- Regression can be **simple** (one input), **multiple** (many inputs) or **polynomial** (curved relationship).

MEMORY TRICK — THE "UNIT TEST"

Ask: **Can I average two answers meaningfully?** Average of ₹40 L and ₹60 L = ₹50 L ✔ → **regression**. Average of "cat" and "dog" = ? ✘ → **classification**. Also: if the answer has **units** (₹, kg, °C, marks) it's regression.

HOW TO WRITE THIS ANSWER FOR FULL MARKS (5–6M)

1. Define both in one line each (mention "supervised").
2. Draw Fig 10.1 (two small sketches: boundary vs line).
3. Write a comparison table with at least 8 rows: objective, output type, question answered, example, algorithms, loss, metrics, threshold.
4. End with the trap: logistic regression = classification.

LIKELY EXAM QUESTIONS — CH 10

1. Differentiate between classification and regression with examples. (5–6M) ★★★
2. Identify whether classification or regression: predict tomorrow's temperature; detect fraud; predict a student's CGPA; classify a tumour as benign/malignant; estimate house price. (2M each) → regression, classification, regression, classification, regression.
3. Why is logistic regression called a classification algorithm? (2M)

11 Linear Regression ★ CIE-1 Q

DEFINITION

Linear regression is a supervised learning and statistical method that models the expected numerical response as a linear combination of one or more predictors (input variables). Its coefficients are commonly estimated by **Ordinary Least Squares (OLS)**. In simple words: *draw the straight line that best fits the data points, then use the line to predict.*

- A statistical model showing the relation between two variables as a linear equation: **Y = mx + c** — "add up the inputs multiplied by some constants to get the output".
- **Regression analysis**: a form of predictive modelling that investigates the relationship between a **dependent** variable (target, y-axis) and **independent** variable(s) (predictors, x-axis); drawing a line through the data points that best fits the overall shape.
- **Origin of the word:** invented by **Francis Galton** (cousin of Charles Darwin). In 1877 he estimated the size of pea seeds from their parents' seeds; with human heights he saw that children of tall parents were tall but less extreme — heights **"regressed" toward the mean**. English meaning of regression: "a return to a former or less developed state" — unrelated to prediction. Statistical meaning: a measure of the relation between the mean value of one variable (output) and the corresponding values of other variables (e.g. time and cost).

11.1 Explanation with an example

Let x = hours studied and y = examination marks. A straight-line model estimates the **expected change in marks for a one-hour increase** in study time. The **intercept** (b_0) is the predicted value when x = 0; the **slope** (b_1) is the predicted change in y per unit of x.

POSITIVE RELATIONSHIP — Y = MX + C

Speed vs distance (time constant): y = distance travelled in a fixed time (dependent), x = speed (independent), m = positive slope, c = y-intercept. As x increases, y increases.

NEGATIVE RELATIONSHIP — Y = −MX + C

Speed vs time (distance constant): y = time taken to travel a fixed distance, x = speed, m = negative slope, c = y-intercept. As x increases, y decreases.

Understanding the regression line: plot the actual data points; draw a line and find the **error** between each estimated point and the actual point; the best-fit line is the one with the least error (least total squared distance between estimated and actual points).

11.2 Mathematical formulation ★ Very likely

Simple linear regression

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

β_0 = intercept, β_1 = slope, ε_i = error (noise) term

Fitted line & residual

$$\hat{y}_i = b_0 + b_1 x_i \qquad e_i = y_i - \hat{y}_i$$

b_0, b_1 are the estimates of β_0, β_1; e_i is the residual

Multiple linear regression: $y_i = \beta_0 + \Sigma_{j=1..p}\, \beta_j x_{ij} + \varepsilon_i$ or in matrix form $\mathbf{y = X\beta + \varepsilon}$

Least-squares cost and derivation of the normal equations

Sum of squared errors: $SSE = \Sigma_{i=1..n} (y_i - b_0 - b_1 x_i)^2$ (cost used in ML: $J(b_0, b_1) = \frac{1}{2n} \Sigma (\hat{y}_i - y_i)^2$)

Minimize: set partial derivatives to zero:

$\partial SSE/\partial b_0 = -2 \Sigma (y_i - b_0 - b_1 x_i) = 0$ $\partial SSE/\partial b_1 = -2 \Sigma x_i (y_i - b_0 - b_1 x_i) = 0$

Solving these two **normal equations** gives:

$$b_1 = \frac{\Sigma(x_i - \bar{x})(y_i - \bar{y})}{\Sigma(x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} \qquad b_0 = \bar{y} - b_1 \bar{x}$$

Shortcut: $S_{xy} = \Sigma xy - (\Sigma x)(\Sigma y)/n$, $S_{xx} = \Sigma x^2 - (\Sigma x)^2/n$. Equivalent: $m = [n\Sigma xy - \Sigma x \Sigma y] / [n\Sigma x^2 - (\Sigma x)^2]$, $c = \bar{y} - m\bar{x}$.

Multiple regression: $\mathbf{b = (X^T X)^{-1} X^T y}$ (when $X^T X$ is invertible).

11.3 Worked example 1: study hours → marks ★ Very likely

Student	Hours (x)	Marks (y)	x^2	xy
1	1	40	1	40
2	2	50	4	100
3	3	65	9	195
4	4	70	16	280
5	5	85	25	425
Total	**15**	**310**	**55**	**1040**

Step 1 — means: $\bar{x} = 15/5 = 3$, $\bar{y} = 310/5 = 62$

Step 2 — centred sums:

$S_{xy} = 1040 - (15 \times 310)/5 = 1040 - 930 = 110$

$S_{xx} = 55 - 15^2/5 = 55 - 45 = 10$

Step 3 — slope & intercept:

$b_1 = 110/10 = \mathbf{11}$, $b_0 = 62 - 11(3) = \mathbf{29}$

Fitted line: $\hat{y} = 29 + 11x$

Interpretation: **+11 marks per extra study hour**.

Step 4 — predict 6 hours: $\hat{y}(6) = 29 + 66 = \mathbf{95}$ (extrapolation beyond 1–5 h — use with care)

Step 5 — fitted values and residuals

x	Observed y	Predicted ŷ	Residual e	e^2
1	40	40	0	0
2	50	51	−1	1
3	65	62	3	9
4	70	73	−3	9
5	85	84	1	1
Total (SSE)				**20**

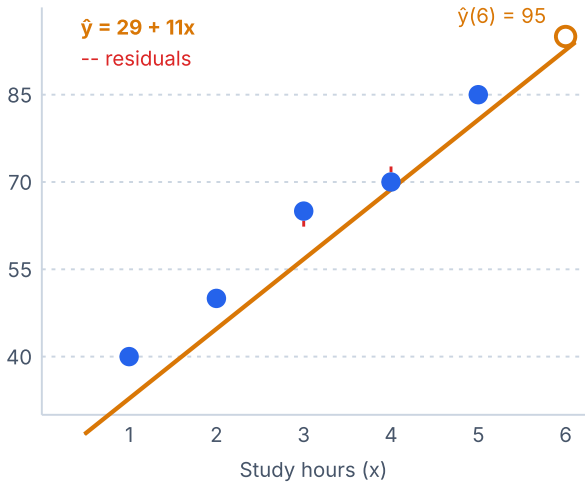


Fig 11.1 Observations (blue), fitted line (orange), residuals (red).

SSE = 20; MSE = 20/5 = **4**; RMSE = √4 = **2**
SST = Σ(y − 62)² = 484 + 144 + 9 + 64 + 529 = **1230**
$R^2 = 1 - SSE/SST = 1 - 20/1230 =$ **0.9837** (98% of the variation explained — excellent fit)

11.4 Worked example 2: the slide's "R² = 0.3" example (poor fit)

x	y	x − x̄	y − ȳ	(x−x̄)(y−ȳ)	(x−x̄)²
1	3	−2	−0.6	1.2	4
2	4	−1	0.4	−0.4	1
3	2	0	−1.6	0	0
4	4	1	0.4	0.4	1
5	5	2	1.4	2.8	4
x̄ = 3, ȳ = 3.6 · Totals				**4**	**10**

m = 4/10 = **0.4**, c = 3.6 − 0.4 × 3 = **2.4** → **y = 0.4x + 2.4**
Predicted y_p: 2.8, 3.2, 3.6, 4.0, 4.4
$\Sigma(y_p - \bar{y})^2 = 0.64 + 0.16 + 0 + 0.16 + 0.64 = 1.6$
$\Sigma(y - \bar{y})^2 = 0.36 + 0.16 + 2.56 + 0.16 + 1.96 = 5.2$
$R^2 = \Sigma(y_p - \bar{y})^2 / \Sigma(y - \bar{y})^2 = 1.6/5.2 =$ **0.31 ≈ 0.3**
→ **not a good fit**: actual and estimated points are far apart.

GOODNESS OF FIT — R² (COEFFICIENT OF DETERMINATION)

R² is a statistical measure of how close the data are to the fitted regression line (also called the coefficient of multiple determination). Two equivalent formulas for a least-squares line:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\Sigma(y - \hat{y})^2}{\Sigma(y - \bar{y})^2} = \frac{\Sigma(\hat{y} - \bar{y})^2}{\Sigma(y - \bar{y})^2}$$ (slide: "distance predicted − mean vs actual − mean")

R² ranges 0 to 1: close to 1 = good fit; close to 0 = poor fit. (Check: example 1 → 1210/1230 = 0.9837 by the second formula too ✔.)

11.5 Worked example 3: salary vs experience (Unit 3 notes)

Experience (years)	Salary (₹ lakhs)
1	3
2	4
3	5
4	6
5	7

$\bar{x} = 3$, $\bar{y} = 5$; $S_{xy} = (-2)(-2) + (-1)(-1) + 0 + 1 + 4 = 10$; $S_{xx} = 10$
$b_1 = 1$, $b_0 = 5 - 3 = 2 \rightarrow$ **y = 2 + x**
Perfectly linear ($R^2 = 1$): salary rises at a constant ₹1 lakh per year of experience. Prediction for 6 years: ₹8 lakhs.

11.6 Evaluation measures

$$MAE = \frac{1}{n}\Sigma|y_i - \hat{y}_i| \quad MSE = \frac{1}{n}\Sigma(y_i - \hat{y}_i)^2 \quad RMSE = \sqrt{MSE} \quad R^2 = 1 - SSE/SST$$

Accuracy / goodness of fit are measured by loss, R², adjusted R², RMSE, etc.

11.7 Gradient descent (iterative alternative to the normal equations)

$$b_0 \leftarrow b_0 - \alpha \frac{1}{n} \Sigma (\hat{y}_i - y_i) \qquad b_1 \leftarrow b_1 - \alpha \frac{1}{n} \Sigma (\hat{y}_i - y_i)\, x_i$$

α = learning rate. Start with any b_0, b_1; repeat the updates; each step moves "downhill" on the cost J until it converges to the least-squares solution.

11.8 Assumptions, limitations, selection criteria, uses

ASSUMPTIONS

- Linear conditional mean (relationship is linear)
- Zero conditional mean of errors
- No perfect multicollinearity among predictors
- Independent errors (for standard inference)
- Constant error variance — homoscedasticity (for conventional standard errors)
- Normal errors mainly needed for exact small-sample inference

LIMITATIONS

- Sensitive to influential **outliers**
- May fail to capture **nonlinear** relationships
- Unreliable **extrapolation**
- Cannot establish **causality** from association alone
- Missing values can remove useful data points

Selection criteria: use when predicting a continuous variable (sales per day, temperature of a city); **not good for classification**; not computationally expensive compared with decision trees etc.; comprehensible and transparent (simple notation).
Uses of regression analysis: (1) determining the strength of predictors (sales vs marketing, age vs income); (2) forecasting an effect — how much y changes per unit change in x ("how much extra sales income for each $100 spent on marketing?"); (3) trend forecasting (price of bitcoin in 6 months).
Where linear regression is used: evaluating trends and sales forecasts (monthly sales on y-axis vs time on x-axis, use the slope to forecast); analysing the impact of price changes (quantity sold vs price); risk assessment in financial services and insurance (number of claims per customer vs age).

11.9 Linear vs polynomial regression

$$\text{Linear: } y = b_0 + b_1 x$$

"Fit a straight line."

$$\text{Polynomial (degree 2): } y = b_0 + b_1 x + b_2 x^2$$

$$\text{(degree 3: } \ldots + b_3 x^3\text{)}$$

"Create powers of the input and fit a curve."

Both predict a continuous value; the difference is the **shape** of relationship they can model. Polynomial regression is still *linear in the coefficients*, so it is solved with the same least-squares method after creating x^2, x^3 features. **Example from notes:** X = 1…8, y = 3, 5, 9, 15, 23, 33, 45, 59 (curved). A straight line gives $\hat{y} = -12 + 8x$ (predicts −4 at x = 1 — bad), while degree 2 fits exactly: $\hat{y} = 3 - x + x^2$.

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1); y = np.array([3,5,9,15,23,33,45,59])
lin = LinearRegression().fit(X, y)                        # straight line
X_poly = PolynomialFeatures(degree=2).fit_transform(X)    # adds 1, x, x^2 columns
poly = LinearRegression().fit(X_poly, y)                  # curve
```

HOW TO WRITE "EXPLAIN LINEAR REGRESSION WITH AN EXAMPLE" (10M)

1. Definition + purpose (predict continuous y from x) + dependent/independent variables.
2. Equation $y = b_0 + b_1x + \varepsilon$ with each symbol explained; residual.
3. Least-squares idea + the b_1, b_0 formulas (derivation if 10M).
4. **Numerical example**: table with x, y, x^2, xy → means → S_{xy}, S_{xx} → b_1, b_0 → line → one prediction.
5. Draw the scatter plot with the line.
6. Goodness of fit (R^2), assumptions, applications, limitations.

LIKELY EXAM QUESTIONS — CH 11

1. Explain linear regression with an example. (10M) ★★★
2. Derive the least-squares estimates of slope and intercept. (6M)
3. Given data, fit a regression line and predict y for a new x; compute R^2. (6–8M) ★
4. What is R^2? Interpret a value of 0.98 and 0.3. (3M)
5. Differentiate linear and polynomial regression. (4M)

12 Naive Bayes Classifier ★ CIE-1 Q

DEFINITION

A **Naive Bayes classifier** is a supervised machine-learning algorithm that predicts the class of an instance using probability. It applies **Bayes' theorem** to estimate how likely each class is given the observed features, and selects the class with the **highest posterior probability**. It is called **naive** because it assumes the features are conditionally independent given the class — once the class is known, one feature tells us nothing about another. This simplifies calculation, although real features may be related.

Popular for: spam email detection, text/document classification, sentiment analysis, news classification, medical diagnosis, recommendation and filtering systems. Example: "Congratulations! You won a free prize." — NB looks at evidence (contains "free"? "won"? "prize"?), computes the probability of each class and picks the highest. **Basic question:** *given the observed features, what is the probability that this observation belongs to a particular class?*

12.1 Bayes' theorem (the engine)

$$P(C \mid X) = \frac{P(X \mid C) \cdot P(C)}{P(X)} \qquad \text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Term	Meaning	Email example
C	Class being considered	Spam
X	Observed features / evidence	Email contains "offer"
$P(C)$	**Prior** — probability of the class before observing evidence	Overall proportion of spam emails
$P(X\|C)$	**Likelihood** — probability of the evidence within that class	Probability that a spam email contains "offer"
$P(X)$	**Evidence** — overall probability of the evidence	Probability that any email contains "offer"
$P(C\|X)$	**Posterior** — probability of the class after observing the evidence	Probability the email is spam given "offer"

12.2 How Naive Bayes classifies an instance ★ Very likely

Instance $X = (x_1, x_2, \ldots, x_n)$. Naive (conditional independence) assumption: $P(X \mid C) = \prod_{i=1..n} P(x_i \mid C)$

So $P(C \mid X) = P(C) \prod P(x_i \mid C) / P(X)$. $P(X)$ is the same for all classes → compare only the **unnormalized scores**:

$$\text{Score}(C) = P(C) \prod_{i=1..n} P(x_i \mid C) \qquad \hat{C} = \text{argmax}_C \left[P(C) \prod P(x_i \mid C) \right]$$

This is the **Maximum A Posteriori (MAP)** decision.

Why "naive"? An email with X_1 = contains "free", X_2 = "prize", X_3 = "winner" — these words are clearly related, yet NB approximates $P(X_1, X_2, X_3 \mid C) \approx P(X_1|C)\, P(X_2|C)\, P(X_3|C)$. The assumption is often unrealistic — hence "naive" — but surprisingly it works very well, especially for text classification.

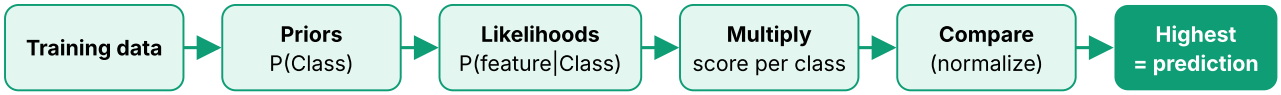


Fig 12.1 The complete Naive Bayes process — use these 6 steps in every numerical.

12.3 How Naive Bayes handles uncertainty

Uncertainty = the evidence doesn't identify the class with complete certainty (some spam contains "offer", some doesn't; some legitimate mail also contains "offer"; an email may have both promotional and ordinary words). NB assigns **probabilities** instead of hard rules: **before** features → priors from training data; **after** features → likelihoods update the class probabilities; **conflicting evidence** → combined probabilistically (a feature supporting Spam is balanced against one supporting Not Spam); **decision** → usually the class with the highest posterior. E.g. P(Spam|X) = 0.80, P(Not Spam|X) = 0.20 → predicts Spam but still assigns 20% to Not Spam; 0.51 vs 0.49 would be far less decisive. Limitation: probabilities depend on the training data and the independence assumption — NB can be overconfident, so its probabilities are not perfectly calibrated measures of certainty.

12.4 Worked example A — email classification (Bernoulli NB) ★ Very likely

Features: x_1 = does the email contain "offer"? x_2 = does it contain "meeting"? (Bernoulli NB: each feature present/absent.)

Email	"offer"	"meeting"	Class
E1	Yes	No	Spam
E2	Yes	No	Spam
E3	Yes	No	Spam
E4	Yes	Yes	Spam
E5	No	No	Spam
E6	No	Yes	Not Spam
E7	No	Yes	Not Spam
E8	No	Yes	Not Spam
E9	Yes	Yes	Not Spam
E10	No	No	Not Spam

New email: contains "offer", does not contain "meeting" → X = (offer = Yes, meeting = No).

Probability	Spam	Not Spam
Prior	5/10 = 0.5	5/10 = 0.5
P(offer = Yes | class)	4/5 = 0.8	1/5 = 0.2
P(meeting = No | class)	4/5 = 0.8	1/5 = 0.2

Step 3 — scores: Score(Spam) = 0.5 × 0.8 × 0.8 = **0.32**; Score(Not Spam) = 0.5 × 0.2 × 0.2 = **0.02** → 0.32 > 0.02 ⇒ **Spam**

Step 4 — normalize: sum = 0.34 → P(Spam|X) = 0.32/0.34 = **0.9412**, P(Not Spam|X) = 0.02/0.34 = 0.0588 (check: sum = 1 ✔)

Step 5 — how belief updates: no word evidence → 50%; "offer" present → (0.5 × 0.8)/(0.5 × 0.8 + 0.5 × 0.2) = 80%; "offer" present and "meeting" absent → 94.12%.

WHAT IF EVIDENCE CONFLICTS? (OFFER = YES, MEETING = YES)

Score(Spam) = 0.5 × 0.8 × 0.2 = 0.08; Score(Not Spam) = 0.5 × 0.2 × 0.8 = 0.08 → P = 0.5 / 0.5. The model has **no preference** — a system may apply a tie-breaking rule or flag the email for human review. NB can represent uncertainty even when evidence doesn't support a clear decision.

12.5 The zero-frequency problem & Laplace smoothing ★ Very likely

If a feature value never appears with a class in training, its likelihood is **0** — e.g. P(offer = Yes | Not Spam) = 0 — and because NB **multiplies** likelihoods, the whole score becomes 0 regardless of all other evidence. Fix: **Laplace (add-one) smoothing** — add 1 to each feature-value count:

$$P(x_i = v \mid C) = \frac{\text{Count}(x_i = v, C) + 1}{\text{Count}(C) + k} \quad (k = \text{number of possible values of the feature; } k = 2 \text{ for binary})$$

P(offer = Yes | Spam) = (4 + 1)/(5 + 2) = 5/7; P(meeting = No | Spam) = 5/7; P(offer = Yes | Not Spam) = (1 + 1)/(5 + 2) = 2/7; P(meeting = No | Not Spam) = 2/7
Score(Spam) = 0.5 (5/7)(5/7) = 25/98; Score(Not Spam) = 0.5 (2/7)(2/7) = 4/98 → P(Spam|X) = 25/29 = **86.21%** — still Spam; smoothing avoids zeros and moderates extreme estimates from small datasets.

12.6 Worked example B — 100 emails and the word "free"

	Spam	Not Spam
Total emails	40	60
Contains "free"	30	6
Doesn't contain "free"	10	54

P(Spam) = 40/100 = 0.4, P(NotSpam) = 0.6
P(Free|Spam) = 30/40 = 0.75, P(Free|NotSpam) = 6/60 = 0.1
P(Free) = (0.75)(0.4) + (0.1)(0.6) = 0.30 + 0.06 = 0.36
P(Spam|Free) = (0.75 × 0.4)/0.36 = 0.30/0.36 = **0.8333 = 83.33%** → classify as Spam

12.7 Worked example C — student pass/fail (categorical NB, with a zero!)

Student	Hours	Attendance	Result
1	Low	Low	Fail
2	Low	Low	Fail
3	Low	High	Fail
4	High	Low	Pass
5	High	High	Pass
6	High	High	Pass
7	High	High	Pass

New student: Hours = High, Attendance = High
P(Pass) = 4/7; P(Hours=High|Pass) = 4/4 = 1; P(Att=High|Pass) = 3/4
Score(Pass) = 4/7 × 1 × 3/4 = 3/7 = **0.4286**
P(Fail) = 3/7; P(Hours=High|Fail) = 0/3 = **0** (zero frequency!); P(Att=High|Fail) = 1/3
Score(Fail) = 0 → prediction **PASS**
With Laplace (k = 2): Pass: 4/7 × 5/6 × 4/6 = 0.317; Fail: 3/7 × 1/5 × 2/5 = 0.034 → still Pass (≈ 90%). The sklearn `CategoricalNB` code in the notes outputs `[1]` = PASS (it applies smoothing by default, alpha = 1).

12.8 Worked example D — species M vs H (10-mark question from slides)

No	Colour	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

P(·|class)	M	H
Colour = Green	2/4	1/4
Legs = 2	1/4	4/4
Height = Tall	1/4	2/4
Smelly = No	1/4	3/4

New: (Green, 2, Tall, No). $P(M) = P(H) = 4/8 = 0.5$

$\text{Score}(M) = 0.5 \times 2/4 \times 1/4 \times 1/4 \times 1/4 = \mathbf{0.0039}$

$\text{Score}(H) = 0.5 \times 1/4 \times 1 \times 2/4 \times 3/4 = \mathbf{0.0469}$

$\Rightarrow$ **Species H** ($P \approx 0.0469/0.0508 = 92.3\%$)

See also the PlayTennis (→ No, 0.795) and stolen-car (→ Not stolen) examples in Chapter 8.7 — those are the two most commonly asked NB numericals.

12.9 Types of Naive Bayes

Type	Feature representation	Example
Gaussian NB	Continuous numerical features, modelled with a Gaussian (normal) distribution within each class	Classifying iris flowers by petal measurements
Multinomial NB	Counts, such as word frequencies	Classifying documents by word counts
Bernoulli NB	Binary features: present / absent	Emails by whether particular words occur
Categorical NB	Features with discrete categories	Weather: sunny / cloudy / rainy; Low / High

12.10 Advantages, limitations and comparisons

ADVANTAGES

- **Simple** to understand and implement
- **Fast** to train and predict
- Works well with **high-dimensional** data (thousands of words)
- Needs relatively **little training data**
- Often very effective for **text classification**: spam filtering, sentiment analysis, news categorization, document classification
- Produces **class probability** estimates

LIMITATIONS

- **Independence assumption** — biggest limitation ("machine" and "learning" are related, NB treats them as independent)
- **Correlated features** can count similar evidence twice
- **Zero-frequency problem** → needs Laplace smoothing
- Probability estimates may be **overconfident / poorly calibrated**
- Performance depends on whether the chosen likelihood model suits the data

Feature	Naive Bayes	Logistic Regression	SVM
Type / model	Classification; generative, based on Bayes' theorem	Classification; discriminative model	Classification; max-margin
Main assumption	Conditional independence	No NB independence assumption	—
Probability output	Yes	Yes	Not inherently
Training speed	Very fast	Fast	Can be slower
Text classification	Very effective, very common	Also widely used	Also effective
Small datasets	Often useful	Can work well	Good (small/medium)
Correlated features	Problematic	Less restrictive	—
Feature scaling	Usually not required	Often helpful	Important
Nonlinear boundary	Not via kernels	No (linear)	Yes, using kernels
Main variants / concept	Gaussian, Multinomial, Bernoulli	Binary, multiclass	Maximum margin

HOW TO WRITE "EXPLAIN THE NAIVE BAYES CLASSIFIER" (10M)

1. Definition + why "naive" (conditional independence).
2. Bayes' theorem with prior/likelihood/evidence/posterior labelled.
3. NB formula: $\hat{C} = \text{argmax } P(C) \prod P(x_i|C)$ (MAP; P(X) dropped).
4. A worked numerical (email or PlayTennis): priors table → likelihood table → scores → decision → normalize.
5. Zero-frequency + Laplace smoothing.
6. Types, advantages, limitations, applications.

LIKELY EXAM QUESTIONS — CH 12

1. Explain the Naive Bayes classifier with an example. (10M) ★★★
2. Why is it called "naive"? What is the zero-frequency problem and how is it solved? (4M)
3. Classify the given instance using NB (PlayTennis / email / stolen car / species). (8–10M) ★
4. How does Naive Bayes handle uncertainty? (4M)
5. List the types, advantages and limitations of NB. (5M)

60-SECOND RECAP — UNIT III CORE

Classification → category; **regression** → number. LR: $\hat{y} = b_0 + b_1x$; $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1\bar{x}$; hours/marks → $\hat{y} = 29 + 11x$, $R^2 = 0.9837$. NB: $\hat{C} = \text{argmax } P(C)\prod P(x_i|C)$; email → Spam 94.12% (86.21% with Laplace).

13 Safety Net: Logistic Regression, SVM, KNN

Not in the CR's list, but they are the other Unit III classification algorithms in your notes. One page each — enough to write a 5-mark answer if the paper surprises you.

13.1 Logistic Regression

Supervised algorithm for **classification** (Pass/Fail, Spam/Not, Disease/No, Fraud/Not, Buy/Not). It predicts the **probability** that an observation belongs to a class. **Why not linear regression?** Its outputs (−0.5, 1.5, 2.3...) are not valid probabilities (must be 0 ≤ P ≤ 1). So we pass the linear combination through the **sigmoid**:

$$z = b_0 + b_1x_1 + \ldots + b_nx_n \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

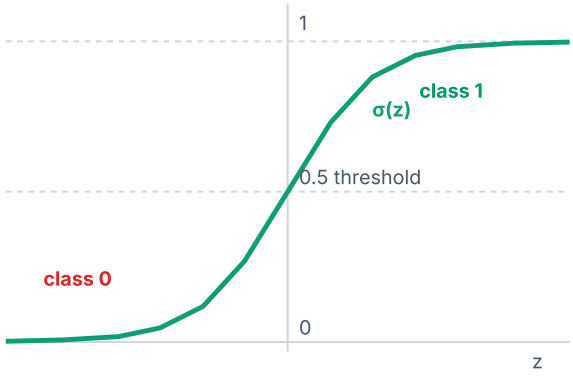


Fig 13.1 The S-shaped sigmoid squashes any z into (0, 1).

Steps: (1) linear combination z → (2) sigmoid → probability 0...1 → (3) threshold (usually 0.5): P ≥ 0.5 → y = 1 else 0 → (4) class. **Example:** hours = 5 → z = 1.2 → σ(1.2) = 0.768 > 0.5 → PASS (predict_proba might give [[0.05, 0.95]]). **Decision boundary:** P = 0.5 ⇔ z = 0 ⇔ $b_0 + b_1x = 0$ (a point); for 2 features a line, 3+ features a hyperplane. **Binary** (2 classes) vs **multiclass** (Iris: Setosa/Versicolor/Virginica). sklearn parameters: **C** = inverse regularization strength (smaller C → stronger regularization), **max_iter**, **solver** ("lbfgs").

Linear vs logistic: linear answers "How much?" (salary, house price, temperature); logistic answers "Which class?" (pass/fail, spam, fraud, disease).

13.2 Support Vector Machine (SVM)

Supervised algorithm mainly for classification (also regression — SVR). **Central idea:** find the decision boundary that separates the classes with the maximum possible margin from the data points.

- **Hyperplane:** $w_1x_1 + w_2x_2 + b = 0$ (e.g. $2x_1 + x_2 - 10 = 0$). 2 features → line, 3 → plane, >3 → hyperplane. General: $w \cdot x + b = 0$; classify by $\text{sign}(w \cdot x + b)$ with labels +1/−1.
- **Margin:** distance between the boundary and the closest points of each class; SVM maximizes it. Margin boundaries $w \cdot x + b = +1$ and -1; **margin width = $2/\|w\|$**. Distance of a point from the hyperplane = $|w \cdot x + b| / \|w\|$.
- **Support vectors:** the training points closest to the boundary (they satisfy $y_i(w \cdot x_i + b) = 1$); they "support"/determine the hyperplane — moving them moves the boundary, removing far points doesn't.

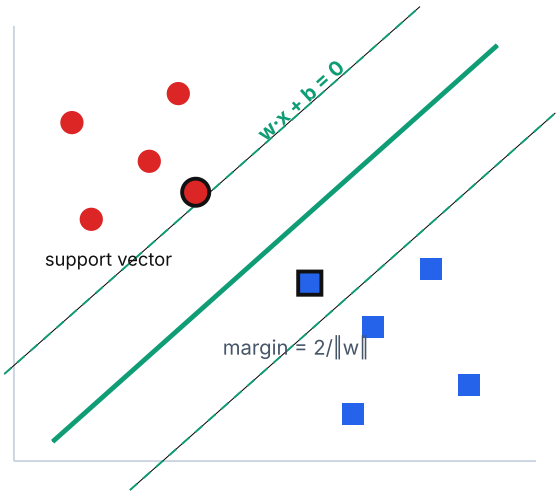


Fig 13.2 Maximum-margin hyperplane; outlined points are support vectors.

- **Hard margin:** assumes perfect separation: minimize $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$. **Soft margin:** allows some points inside the margin or misclassified (noise, outliers, overlap) — controlled by **C**: large C = try hard to classify all training points (small margin, complex boundary, may overfit); small C = allow mistakes for a wider margin (simpler, better generalization).
- **Kernel trick:** builds nonlinear boundaries without explicitly transforming the data. Kernels: linear, polynomial (degree), **RBF** $K(x, x') = \exp(-\gamma\|x - x'\|^2)$, sigmoid. **γ**: small → each point influences a large area → smoother boundary; large → small influence → complex boundary.
- **Feature scaling** is important (age 18–60 vs salary 20,000–2,00,000): use StandardScaler — `fit_transform` on train, `transform` on test (avoid data leakage).
- **Pros:** effective in high dimensions (text, image), maximum-margin principle, kernel trick, good for small/medium datasets. **Cons:** slow on very large data, needs scaling, sensitive to C/γ/kernel/degree, less interpretable. Application: network intrusion detection (normal vs attack).

13.3 K-Nearest Neighbors (KNN)

Supervised algorithm for classification *and* regression: a new point gets the class of its nearby points — "*tell me who your nearest neighbours are, and I will tell you your class*". **K** = number of neighbours considered; **nearest** = closest by distance; **neighbours** = those closest points.

$$\text{Euclidean } d = \sqrt{[(x_1-y_1)^2 + (x_2-y_2)^2 + \ldots]}$$
$$\text{Manhattan } d = \Sigma|x_i - y_i|$$

Algorithm: (1) compute the distance from the new point to every training point; (2) pick the K nearest; (3) **majority vote** (classification) or **average** (regression, e.g. houses ₹50, 55, 60 L → ₹55 L).

Choosing K: K = 1 → captures very local patterns but sensitive to noise/outliers → **overfitting**; very large K (50, 100) → very smooth boundary, ignores local patterns → **underfitting**; moderate K (≈5) balances. Choose K by cross-validation on the training set, not by test accuracy.

Lazy / instance-based learning: KNN learns no equation (no b_0, b_1); training just **stores** the data; the work happens at prediction (distances → K neighbours → vote). Needs **feature scaling** (salary would dominate age). Parameters: n_neighbors, weights ('uniform' or 'distance' — closer neighbours count more), metric (euclidean, manhattan, minkowski).

Pros: very simple, easy to implement, no real training, nonlinear boundaries, classification + regression. **Cons:**

NUMERICAL (ILLUSTRATIVE)

Student	Hours x_1	Attend. x_2 (/10)	Class	d to (6, 7)
A	2	4	Fail	5.00
B	3	5	Fail	3.61
C	6	8	Pass	1.00
D	7	7	Pass	1.00
E	8	9	Pass	2.83
F	4	3	Fail	4.47

New student (6, 7). K = 3 → C, D, E → Pass 3, Fail 0 → **Pass**. K = 5 → C, D, E, B, F → Pass 3, Fail 2 → **Pass** (majority vote).

Learner	Training	Prediction
Logistic regression	optimize → learn coefficients	fast (plug into equation)
KNN (lazy)	just store data	slow (distances, vote)

slow prediction, sensitive to scaling, irrelevant features and choice of K, curse of dimensionality. Uses: healthcare (similar patients), recommendation, image classification, pattern recognition, anomaly detection, student analytics.

ONE-LINE ANSWERS

Logistic = sigmoid of a linear score + threshold. SVM = maximum-margin hyperplane defined by support vectors; kernels for nonlinearity; C trades margin vs errors. KNN = vote of the K nearest points; lazy learner; scale your features; small K overfits, large K underfits.

14 Last-Minute Revision

Read this the night before and the morning of the exam. Everything here is explained in full in the earlier chapters.

14.1 Formula & rule sheet

Search (Unit I)	
Greedy best-first	$f(n) = h(n)$
A*	$f(n) = g(n) + h(n)$
Admissible	$h(n) \le h^*(n)$ (never overestimates)
Consistent	$h(n) \le c(n, a, n') + h(n')$
BFS / IDS time	$O(b^d)$; IDS space $O(bd)$
DFS	time $O(b^m)$, space $O(bm)$
Bidirectional	$O(b^{d/2})$
Effective branching factor	$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
Logic (Units I–II)	
Implication	$P \Rightarrow Q \equiv \neg P \vee Q$ (false only when P=T, Q=F)
Biconditional	$P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$
De Morgan	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$; $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
Quantifier negation	$\neg\forall x\ P \equiv \exists x\ \neg P$; $\neg\exists x\ P \equiv \forall x\ \neg P$
Entailment	$KB \models \alpha \Leftrightarrow M(KB) \subseteq M(\alpha) \Leftrightarrow KB \wedge \neg\alpha$ unsat.
Modus Ponens	$\alpha \Rightarrow \beta,\ \alpha \vdash \beta$
Resolution	$P \vee Q,\ \neg Q \vee R \vdash P \vee R$
UI / EI	$\forall v\ \alpha \vdash SUBST(\{v/g\}, \alpha)$; $\exists v\ \alpha \vdash SUBST(\{v/k\}, \alpha)$, k new
Unify	$SUBST(\theta, p) = SUBST(\theta, q)$
Quantifier + connective	$\forall \ldots \Rightarrow \quad \exists \ldots \wedge$

Probability (Unit II)	
Conditional	$P(A\|B) = P(A \wedge B) / P(B)$
Product rule	$P(A \wedge B) = P(A\|B)P(B) = P(B\|A)P(A)$
Sum rule	$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$
Bayes	$P(A\|B) = P(B\|A)P(A) / P(B)$
Total probability	$P(B) = \Sigma\ P(B\|A_i)P(A_i)$
MAP	$h_{MAP} = \text{argmax}\ P(D\|h)P(h)$
Bayes net joint	$P(x_{1..n}) = \prod P(x_i \mid \text{Parents}(X_i))$
Naive Bayes	$v_{NB} = \text{argmax}\ P(v) \prod P(a_i\|v)$
Laplace smoothing	$(\text{count} + 1) / (\text{Count}(C) + k)$
Regression (Unit III)	
Model	$\hat{y} = b_0 + b_1 x;\ e = y - \hat{y}$
Slope	$b_1 = S_{xy}/S_{xx} = \Sigma(x-\bar{x})(y-\bar{y})/\Sigma(x-\bar{x})^2$
Shortcuts	$S_{xy} = \Sigma xy - \Sigma x \Sigma y/n$; $S_{xx} = \Sigma x^2 - (\Sigma x)^2/n$
Intercept	$b_0 = \bar{y} - b_1\bar{x}$
Errors	$MSE = SSE/n$; $RMSE = \sqrt{MSE}$; $MAE = \Sigma\|e\|/n$
R^2	$1 - SSE/SST = \Sigma(\hat{y}-\bar{y})^2/\Sigma(y-\bar{y})^2$
Matrix form	$b = (X^TX)^{-1}X^Ty$
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$
SVM margin	$2/\|w\|$

14.2 Answers you must be able to reproduce from memory

Problem	Final answer
A* Arad → Bucharest (Romania)	Arad–Sibiu–Rimnicu Vilcea–Pitesti–Bucharest, cost **418** (greedy gives 450 via Fagaras)
8-puzzle h1, h2 for 7 2 4 / 5 _ 6 / 8 3 1	h1 = **8**, h2 = **18**
Crime KB	Criminal(West) in 2 FC iterations; θ = {x/West, y/M1, z/Nono}
Unify Q(a, g(x, a), f(y)) & Q(a, g(f(b), a), x)	{x/f(b), y/b}
Meningitis	P(M|S) = **0.0014**
Cancer, positive test	0.0078 vs 0.0298 → h_{MAP} = **¬cancer** (P(cancer|+) = 0.21)
Liver disease / alcoholic	**0.14**
PlayTennis (sunny, cool, high, strong)	0.0053 vs 0.0206 → **No** (0.795)
Stolen car (Red, SUV, Domestic)	0.024 vs 0.072 → **Not stolen**
Rain–Sprinkler–GrassWet	P(W) = **0.448**
Cloudy network	P(W) = **0.6471**; P(S|W) = 0.430 (slide: 0.438)
Find-S EnjoySport	⟨Sunny, Warm, ?, Strong, ?, ?⟩
Candidate-Elimination EnjoySport	S = {⟨Sunny, Warm, ?, Strong, ?, ?⟩}; G = {⟨Sunny, ?, ...⟩, ⟨?, Warm, ?, ...⟩}
Hours → marks regression	$\hat{y}$ = **29 + 11x**; $\hat{y}(6)$ = 95; R^2 = **0.9837**
Email NB (offer = Y, meeting = N)	Spam, **94.12%**; with Laplace **86.21%**

14.3 Top questions by priority

MUST-PREPARE (★★★)

1. PEAS for a given agent + environment properties table
2. Types of agents with diagrams
3. BFS/DFS/UCS/IDS comparison; trace a search on a graph
4. A* algorithm, admissible & consistent heuristics; trace on a map
5. Wumpus world: PEAS, properties, reasoning
6. PL → CNF; prove by resolution
7. English → FOL translations
8. Forward/backward chaining: crime / peanuts
9. Resolution refutation in FOL (John happy)
10. Bayes' theorem + cancer/meningitis MAP
11. Bayesian network definition + numerical
12. Classification vs regression
13. Linear regression with numerical example
14. Naive Bayes with numerical example

ALSO PREPARE (★★)

1. Four approaches to AI; Turing test
2. Rationality vs omniscience; learning and autonomy
3. Problem formulation (8-puzzle, vacuum, 8-queens)
4. Heuristics h1/h2, dominance, relaxed problems
5. Entailment, soundness, completeness
6. Forward vs backward chaining (PL & FOL)
7. Unification algorithm + MGU
8. Knowledge engineering (full adder)
9. Knowledge graphs: triples, ontology, open-world, SPARQL
10. Well-posed learning problems (T, P, E)
11. Types of ML; key tasks; ML process
12. Find-S and Candidate-Elimination numericals
13. Laplace smoothing; why "naive"

14.4 Night-before checklist

☐ I can draw: agent–environment loop, 5 agent architectures, Wumpus grid

☐ I know KG vs ontology, RDF triple, open-world assumption

☐ I can write the BFS / DFS / A* comparison table from memory

☐ I can trace A* on the Romania map and get 418

☐ I can write a truth table for ⇒ and ⇔

☐ I can convert a sentence to CNF and resolve to □

☐ I can translate "all / some / not all / no one" sentences into FOL

☐ I can unify two expressions and give the MGU

☐ I can do the crime example by FC and BC with a proof tree

☐ I can derive Bayes' theorem and solve the cancer MAP problem

☐ I can compute a joint probability and P(W) in a Bayes network

☐ I can run Naive Bayes on PlayTennis (0.0053 vs 0.0206)

☐ I can give T, P, E for three learning problems

☐ I can run Find-S and Candidate-Elimination on EnjoySport

☐ I can write the classification vs regression table (8+ rows)

☐ I can fit $\hat{y} = 29 + 11x$ from the hours/marks table and compute R^2

IN THE EXAM HALL

- Read all questions first; start with the one you know best to build momentum.
- Spend marks-proportional time: about 1.5 min per mark.
- If a numerical goes wrong, still write the formula, the method and the table — method marks add up.
- Leave 5 minutes to underline keywords and box final answers.

All the best for CIE-1!